Measuring Feedback-Driven Improvement in Lightweight Code-Security Verifiers for Spec-Driven Code Generation

PI: Younghun Chae, Department of Computer Science, Kent State University at Stark, Ohio 44720

Cash funding needed: $79,350

Abstract

Does bundled FP/FN-targeted ordering and focal weighting improve a post-hoc code-security verifier over standard supervised fine-tuning (SFT) on identical candidate IDs at matched acquisition and adaptation budgets? The primary endpoint is acquisition cost per accepted detection at first qualification (C/D) for the oracle-supported constructed cohort (generated, CVE-derived, and seeded instances retained under executable evidence) on a fixed audit of witness-bearing defects; expenditure, accepted detections, and qualification checkpoint are also reported separately. An open 30B-A3B LoRA verifier screens spec-driven Python code; executable evidence defines the primary cohort and static-only evidence a secondary cohort. The N=30 pilot was 30/30 buildable and 30/30 reproducible but 0/30 for target detection; causes and the M1 20% dynamic-harness capacity gate are reported. No experimental efficacy results exist yet. Failure at M6 (project month 6) or failure to qualify yields a feasibility release with no efficacy claim. The experiment tests the ordering/weighting bundle, not individual components.

Keywords: LLM-generated code security, spec-driven code generation, execution-oracle verification, fine-tuned security verifier, feedback-driven improvement, secure code generation

Introduction

Spec-driven code generation translates natural-language specifications into code, but security remains uncertain: one study found approximately 40% of generated programs vulnerable [1]. CodeSecEval [2] provides a Python, execution-validated, oracle-labeled corpus; RealSec-bench [3] assesses repository tasks, and security hardening [4] improves generation in specific settings. Our question is how a post-hoc verification layer improves as it accumulates feedback on a frozen generation distribution.

Question and contribution. Shared candidate IDs isolate the effect of bundled FP/FN ordering and focal weighting at matched acquisition and adaptation budgets. The contribution is a controlled test of this bundle against standard LoRA SFT; individual component effects are not identified. The bundle tests one operational policy: ordering allocates exposure to current errors while weighting reallocates token-level effort within each unit, concentrating a fixed update budget on current decision failures.

Related approaches include prompt/memory adaptation (SIVA [7]), hybrid analysis (IRIS [8]), pseudo-labeling (SSVD [9]), sampling and self-paced learning [20], [21], FP triage [10], and generator-detector co-evolution [22]. The Amazon Security CFP [5], [6] alignment is feedback-driven self-improvement, with public benchmarks for spec-driven security and capability-aware escalation recognition supported by the released audit protocol and calibration-fitted confidence thresholds, respectively; secure-code enhancement is secondary. Unlike these, this study tests a post-hoc ordering/weighting bundle with shared candidate IDs, executable labels, and acquisition-cost qualification, without claiming these systems lack execution-oracle validation or matched-budget comparison; AWS code assistants document the residual-vulnerability class screened [23].

Pilot and limitations. An N=30 vulnerable-function pilot generated with Qwen3-Coder-30B-A3B-Instruct was syntax-buildable in 30/30 cases and reproducible across two Semgrep runs in 30/30, but yielded 0/30 target-CWE findings (causes: rule-source mismatch under `p/python` without web-framework request context; safe idioms); dynamic harnesses (Atheris/local fixtures) supply the concrete execution traces the M1 20% capacity gate measures. Manual inspection found vulnerable patterns in 20/30, injection-safe code in 10/30. These are not execution witnesses; executable yield and detector accuracy remain unmeasured.

Methods: Design and Oracle Evidence

Corpus and split. Python is the core scope. Target CWEs are {79, 89, 918, 78, 94, 22, 798, 352, 502, 863}, all in the 2024 Top-25 [13]; membership does not imply KEV presence (CWE-352 has zero KEV-linked CVEs in that table). Sources are MIT-licensed Python repository tasks in SWE-bench format [11], disclosed-CVE vulnerable functions from public Python repositories, and seeded controls. Big-Vul and PrimeVul

[12] are C/C++ references, not corpus inputs. Qwen3-Coder generates all generated instances, limiting generalization to that family; shared model-family blind spots may remain undetected. Pair negatives with the same task/repository and fixtures using repaired or policy-justified safe implementations, keeping pairs in one cluster.

The cluster-disjoint five-way split comprises training, feedback, calibration, non-adapting audit (allocated to pilot and confirmatory subsets), and final holdout. Calibration alone fits thresholds, and confidence. M3 freezes splits and the holdout label-generation path. Cluster connected repositories, forks, shared CVEs, and prompt families; remove near-duplicates. Predictions precede audit-label access: preserve immutable, timestamped predictions at every scheduled checkpoint with model/calibration artifact hashes; a designated audit custodian, separate from model updating, seals labels and scores; the custodian releases only pilot-audit at M6, confirmatory results after the stopping cap and freeze, and final holdout once at M10.

Per-CWE attempted targets are 150 candidates in 15 clusters each for training and feedback, and 50 candidates in 5 clusters each for calibration, pilot-audit, confirmatory audit, and final holdout: 5,000 candidates in 500 clusters, at most ten per cluster. For the seven executable CWEs, planning scenarios at 10/20/30% witness yield before attrition give 350/700/1,050 usable Tier 1 positives from 3,500 attempts overall; these are not achieved or guaranteed counts, and justified negatives have no measured yield. Freeze capacity-contingent allocations at M1/M3 and report planned usable units alongside attempts; reductions need revised power.

Cohorts and feasibility ledger. Only retained executable CWEs 79, 89, 918, 78, 94, 22, and 502 enter the primary fit, using accepted Tier 1 positives and justified negatives. CWEs 798, 352, and 863 form a static-only secondary cohort, excluded from the primary endpoint. For every CWE/split and generated/CVE-derived/seeded provenance, report attempted, buildable, Tier 1, justified-negative, unresolved, and retained positive-bearing cluster counts, witness yield, and two-run reproduction. The estimand is the arm contrast in the oracle-supported constructed cohort, not natural vulnerability prevalence. Provenance is mutually exclusive: seeded, then unseeded CVE-derived, then generated instances. Freeze attempted proportions by CWE/split at M3 and report retained proportions after exclusions; no mix or split-specific yields have been measured.

Oracle battery and acceptance. Run CodeQL/Semgrep for all ten CWEs; use Atheris / Jazzer.js [14] for differential execution. Static-only review requires secret status (CWE-798), browser/session context (CWE-352), or principal/resource policy (CWE-863). Two independent reviewers record raw agreement/kappa before PI adjudication; missing context remains unresolved. M3 freezes witness-share minima and SAST-blind selection; below-minimum classes support only oracle-conditioned screening.

Harness acceptance requires buildable vulnerable/repaired pairs, a policy-violating trace only in the vulnerable version, preserved functionality after repair, and two replays meeting M4 criteria under declared policies and pinned fixtures. Review minimal repairs; tag seeded mutations of safe outputs. Instrument actual effects with local databases, canaries, and sandbox fixtures; a hook invocation or crash alone is insufficient.

Tier 1 witnesses are primary positives despite static silence, deduplicated by span/CWE; Tier 2 static agreement supplies sensitivity-fit weak positives only. Primary negatives require policy/fixture justification. Oracle-silent/failed-harness cases remain unresolved and unsupervised, retaining coverage/attrition/cost accounting. Static agreement cannot independently validate SAST baselines or exclude shared generator/verifier blind spots outside oracle coverage.

Endpoint and Evaluation

Verifier and learning loop. Qwen3-Coder-30B-A3B-Instruct (MoE, 3B active) uses local LoRA/QLoRA. Each candidate/CWE unit inputs a specification, function, and policy and outputs CWE/span alerts with confidence. Train token cross-entropy on canonical alert lists (empty only for justified negatives), averaging tokens within each unit. Match same-CWE spans at token intersection-over-union ≥ 0.50 by maximum one-to-one matching after deduplication. Review every alert and defect inventory under the policy/fixture scope: matched accepted defects are detections, refuted unmatched alerts are false positives, unmatched accepted defects are false negatives, and insufficiently supported alerts are reported separately. A witnessed defect does not refute other alerts in its unit. A prespecified sensitivity analysis counts every unsupported alert

as a false positive and repeats qualification and the C/D comparison. Calibration fits alert confidence to matched Tier 1 status on adjudicated calibration units, including unmatched alerts on justified negatives. M1 records p50/p95 latency, throughput, hardware, and decoding conditions (token caps, batching, quantization, warm/cold). Unlike CoSec [15] and CoSEFA [16], verification is post-hoc with asynchronous auditing.

For batch t, log predictions before label access, reveal and score labels, then update for $t{+}1$. Within each paired seed, fit one initial adapter and copy it to both arms. Arms share acquired IDs, label availability, optimizer, update budget, calibration, and fitting procedures. M3 freezes batch size, steps, replay, and unresolved-slot handling. Each targeted-arm pass orders available training/replay units by pre-update mixed FP/FN, FN-only, FP-only, then remaining resolved status, with seeded within-group permutations, and applies focal token weights $(1 - p_{\text{true}})^{\gamma}$ (p_{true}: correct-target-token probability; γ: M3-fixed), detached from gradients and normalized to mean one per unit (uniform if all zero), averaging units equally. Normalization preserves each unit's total loss weight while shifting token-level gradient toward difficult targets; ordering changes exposure within the fixed budget. Pre-update errors and probabilities are fixed at pass start. SFT uses a seeded permutation and uniform weights. Paired seeds vary adapter initialization, within-group/SFT permutations, replay sampling, and training stochasticity, with matching random streams; acquired IDs, splits, and deterministic decoding remain fixed. Audit labels never adapt weights.

Acquire 1,500 training and 500 calibration candidates before $t = 0$, then ten disjoint seeded-random feedback batches of 150 IDs (15 per CWE); only primary-cohort resolved units enter the primary fit. Unresolved attempts consume slots and cost without replacement. Score each update; stop at ten rounds or common M3 label/compute caps informed by M1, never audit-driven early stopping. Produce audit labels once and reuse without relabeling charges; the custodian seals confirmatory labels and scores until stopping. At all 11 scheduled checkpoints, save both arms' predictions for each seed on both audit subsets. Before unsealing, freeze model checkpoints, calibration artifacts, and prediction-generation rules; determine the first qualifying checkpoint from saved predictions, no regeneration or retuning.

Primary endpoint and accounting. The primary endpoint is acquisition cost per accepted detection at first qualification (C/D) on a fixed confirmatory audit. At each scheduled checkpoint (including $t = 0$), record cumulative acquisition expenditure C, distinct matched Tier 1 detections D, and checkpoint separately. Accepted-witness recall is D divided by all audit Tier 1 defects; let M, R, and U count deduplicated matched, refuted, and unsupported alerts on primary audit units. Primary adjudicated precision excludes unsupported alerts: $P = M/(M + R)$; sensitivity precision includes them as errors: $P_{\text{all}} = M/(M + R + U)$. A zero denominator cannot qualify; report U separately. Qualification requires nonzero alerts, $D > 0$, accepted-witness recall ≥ 0.80, and precision ≥ 0.80, using calibration-fitted thresholds. These planning floors are not deployment safety guarantees. Compare C/D at the first qualifying checkpoint. Report the paired checkpoint difference alongside B.

Cost includes policy/harness synthesis, validation, scans, fuzzing, failed searches, timeouts, retries, and review/repair/replay labor/compute at common M1 rates. Charge shared setup, original acquisition, and audit scoring equally; acquisition C excludes adaptation (initial fits and pre-update scoring) and verifier inference, logged separately; retain seed-specific costs and do not recharge cache hits. Tier 2 and unresolved cases incur cost but never enter D. A secondary end-to-end endpoint adds generation, adaptation, inference, and simulated review under an M3-fixed amortization horizon.

Confirmatory inference. Protocol v1 (`ara-solo-protocol-v1.tex`, included in this package) specifies the simulation grid, replay, and cluster bootstrap (clusters resampled jointly across arms and checkpoints, seeds retained; qualification and arm C/D means recomputed per replicate; intervals target cluster variation conditional on the fitted seed pairs). Simulation validation of the selection rule and interval procedure is a prerequisite for confirmatory inference. Before M3, require at most 5% false useful-benefit claims at $B \leq 0.20$ and at least 95% conditional interval coverage; otherwise freeze a final-scheduled-checkpoint comparison under common caps and validate it by the same criteria. If neither rule validates, inference is descriptive only. All three paired seeds in each arm must qualify; average their C/D values arithmetically. Efficacy also requires at least 95% qualifying bootstrap replicates; report non-qualification frequencies and seed variability. M3 freezes the simulation-supported positive-bearing-cluster minimum; infeasible power yields feasibility-only scope.

Evaluation and sensitivity. Final holdout compares CodeQL, Semgrep, their union, the frozen verifier, and SFT with precision/accepted-witness recall/F1 per executable CWE and overall, and cost per flagged defect. Report static-only results separately. Security metrics condition on buildable, functionally valid, adjudicated code; joint outcomes use every attempted ID (including failures/timeouts/unresolved) and count adjudicated correct decisions on valid code. Report unsupported alerts/all deduplicated alerts separately from adjudicated false positives by arm/checkpoint/CWE. Report residual miss risk and positive yield at fixed per-CWE alert budgets, plus coverage/attrition/cost funnels and trajectories. Generated-only sensitivity requires 100+ resolved generated-only audit units in 20 positive-bearing clusters; below this, report descriptive metrics only. Prequential/audit F1 curves and eligible sensitivity comparisons use cluster-paired intervals and seed variability.

Milestones and Gates

Q1 (M1–3): measure capacity/cost and the 500-instance harness pilot; release oracle pipeline/protocol v1; freeze scope and power plan at M3. The pooled M1 capacity gate requires at least 100 accepted executable positive units across at least 20 positive-bearing clusters, with witness yield at least 20% of preselected executable attempts; failure makes the current executable scope no-go pending a revised allocation/power plan. Q2 (M4–6): apply M4 gate, scale retained classes, release verifier v1/frozen baseline, run M5/M6 gates. Q3 (M7–9): run the paired loop if M6 passes. Q4 (M10–12): open holdout once without reselection, evaluate cost-quality Pareto/sensitivity, and deliver the M12 OSS release (verifier code, oracle-validated corpus where licensing permits, evaluation/audit protocol), Zenodo DOI, and paper submission.

M4 witness/reproduction gate. The fuzz-dependent classes are CWE-79, CWE-918, CWE-94, CWE-22, and CWE-502. Unlike the pooled M1 capacity gate, M4 tests each class on 50 preselected gate attempts separate from confirmatory audit. Each needs at least 10 executable witnesses in 50 preselected attempts (20%) and at least $\lceil 0.95n \rceil$ reproducing witnesses out of n accepted witnesses (10/10 for $n = 10$), with two-run reproduction as the fraction of accepted witnesses whose two scheduled replays both reproduce the policy violation; static-only CWEs need deterministic reproduction. Failures trigger per-cause exclusion or restriction to the static cohort. CWE-89/78 positives use marked seeded injection controls and CVE-derived vulnerable revisions with repairs; generation or mutation alone is not acceptance evidence.

M6 feasibility/non-futility gate. Proceed if retained executable CWEs pass M4, pilot-audit positive-bearing clusters meet the M3-frozen minimum, and the shared-ID label/update/replay pipeline reproduces within caps. No favorable effect is required. Only pilot-audit informs M6; confirmation/holdout cannot guide reselection. Failure yields feasibility artifacts, frozen-baseline evaluation, and simulation sensitivity, with no efficacy claim.

Sealed confirmatory decision. The independent confirmatory audit determines efficacy after the stopping cap. Define $B = 1-(C/D)_{\text{targeted}}/(C/D)_{\text{SFT}}$ and minimum useful effect $\delta = 0.20$ (planning choice). Subject to the qualification requirements above, the 95% paired cluster interval $[L, U]$ supports useful benefit if $L > \delta$, excludes it if $U < \delta$, and is otherwise unresolved. Pilot effects remain descriptive; negative and inconclusive results are published.

Funds Needed

The total cash budget is $79,350, all retained at Kent: graduate wages $34,350 (1,500 h at $20/h, $30,000 base + 14.5% fringe); undergraduate wages $12,000 (800 h at $15/h); PI supervision/adjudication $20,000; hardware/storage/oracle $4,000; two-conference travel $5,000; optional external frontier-judge inference $3,000; contingency $1,000. Estimates, not commitments. Within wages, reserve 300 graduate and 200 undergraduate hours ($9,000 total) for harness construction and validation, leaving 1,200 and 600 hours for other work.

Staffing and capacity risks. Graduate wages include 14.5% fringe; undergraduate fringe, tuition, and other mandatory charges remain excluded; PI hours, full personnel coverage, and GPU model/memory/capacity remain unconfirmed. M3 freezes core scope against confirmed staffing/capacity. If commitments fail, reduce CWEs/candidates before split freeze and revise power; if core scope remains unaffordable, release feasibility artifacts and a frozen-baseline evaluation without an efficacy claim. Up to $40,000 conditional AWS credits cover stretch work only; planned allocations are fixed at M1.

Appendix A — References

[1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proc. 2022 IEEE Symp. Security and Privacy (SP)*, May 2022, pp. 754–768, doi: 10.1109/SP46214.2022.9833571.

[2] J. Wang et al., "Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking," *Proc. ACM Softw. Eng.*, vol. 3, no. ISSTA, Art. ISSTA068, Oct. 2026, doi: 10.1145/3832159.

[3] Y. Wang, Z. Zhang, C. Wang, X. Xu, M. Liu, Y. Wang, J. Chen, and Z. Zheng, "RealSec-bench: A Benchmark for Evaluating Secure Code Generation in Real-World Repositories," in *Findings Assoc. Comput. Linguistics: ACL 2026*, Jul. 2026, pp. 35866–35883, doi: 10.18653/v1/2026.findings-acl.1786.

[4] J. He and M. Vechev, "Large Language Models for Code: Security Hardening and Adversarial Testing," in *Proc. 2023 ACM SIGSAC Conf. Computer and Communications Security (CCS '23)*, Nov. 2023, pp. 1865–1879, doi: 10.1145/3576915.3623175.

[5] "Amazon Security Call for Proposals, Fall 2026," Amazon Research Awards. Accessed: Oct. 1, 2026. [Online]. Available: https://amazon.science/research-awards/call-for-proposals/amazon-security-call-for-proposals-fall-2026

[6] "Frequently Asked Questions," Amazon Research Awards. Accessed: Oct. 27, 2026. [Online]. Available: https://amazon.science/research-awards/frequently-asked-questions

[7] V. Walischewski, G. Zizzo, and K. Webster, "SIVA: Self-Improving Vulnerability Agent," presented at the NeurIPS 2025 Workshop on Reliable ML from Unreliable Data, Dec. 2025. [Online]. Available: https://research.ibm.com/publications/siva-self-improving-vulnerability-agent

[8] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," in *Proc. Int. Conf. Learning Representations (ICLR 2025)*, 2025. [Online]. Available: https://iclr.cc/virtual/2025/poster/30698

[9] X. Yu, G. Lin, X. Hu, J. W. Keung, and X. Xia, "Less Is More: Unlocking Semi-Supervised Deep Learning for Vulnerability Detection," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Art. 62, Feb. 2025, doi: 10.1145/3699602.

[10] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," *Proc. ACM Softw. Eng.*, vol. 3, no. ISSTA, Art. ISSTA009, pp. 197–220, Oct. 2026, doi: 10.1145/3832100.

[11] "SWE-bench," GitHub repository, Princeton NLP. Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/princeton-nlp/SWE-bench.

[12] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, "Vulnerability Detection with Code Language Models: How Far Are We?" in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE 2025)*, May 2025, doi: 10.1109/ICSE55347.2025.00038. (PrimeVul; C/C++ benchmark, excluded as a corpus input in this proposal, CWE-family reference only.)

[13] "2024 CWE Top 25 Most Dangerous Software Weaknesses," MITRE Common Weakness Enumeration, full ranked table including CVEs in KEV. Accessed: Nov. 2, 2026. [Online]. Available: `https://cwe.mitre.org/top25/archive/2024/2024_top25_list.html`

[14] "OSS-Fuzz: Integrating a Python Project; Integrating a JavaScript Project," Google, language integration documentation. Accessed: Nov. 3, 2026. [Online]. Available: `https://google.github.io/oss-fuzz/getting-started/new-project-guide/python-lang/`; `https://google.github.io/oss-fuzz/getting-started/new-project-guide/javascript-lang/`

[15] D. Li, M. Yan, Y. Zhang, Z. Liu, C. Liu, X. Zhang, T. Chen, and D. Lo, "CoSec: On-the-Fly Security Hardening of Code LLMs via Supervised Co-decoding," in *Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA 2024)*, Sept. 2024, doi: 10.1145/3650212.3680371.

[16] X. He, D. Li, H. Wen, Y. Zhu, C. Liu, M. Yan, and H. Zhang, "CoSEFA: An LLM-Based Programming Assistant for Secure Code Generation via Supervised Co-Decoding," in *Proc. ACM Int. Conf. Foundations of Software Engineering (FSE Companion '25)*, Jun. 2025, pp. 1198–1202, doi: 10.1145/3696630.3728609.

[17] J. Jung, F. Brahman, and Y. Choi, "Trust or Escalate: LLM Judges with Provable Guarantees for Human Agreement," in *Proc. Int. Conf. Learning Representations (ICLR 2025)* (Oral), 2025. [Online]. Available: https://iclr.cc/virtual/2025/oral/31838

[18] C. Spiess, D. Gros, K. S. Pai, M. Pradel, M. R. I. Rabin, A. Alipour, S. Jha, P. Devanbu, and T. Ahmed, "Calibration and Correctness of Language Models for Code," in *Proc. IEEE/ACM 47th Int. Conf. Software Engineering (ICSE 2025)*, Apr. 2025, doi: 10.1109/ICSE55347.2025.00040.

[19] "HackerOne Code," HackerOne. Accessed: Nov. 4, 2026. [Online]. Available: https://www.hackerone.com/product/code

[20] X. Yang, S. Wang, Y. Li, et al., "Does Data Sampling Improve Deep Learning-Based Vulnerability Detection? Yeas! and Nays!" in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering (ICSE 2023)*, May 2023, pp. 2287–2298, doi: 10.1109/ICSE48619.2023.00192.

[21] Z. Cheng, Y. Yang, H. Zhang, L. Yang, J. Hu, J. Xu, B. Liu, and H. Shen, "Leveraging Self-Paced Learning for Software Vulnerability Detection," *ACM Trans. Softw. Eng. Methodol.*, advance online publication, pp. 1–26, Aug. 28, 2026, doi: 10.1145/3844141.

[22] M. R. Hasan and H. Bagheri, "Self-Discovering Security Oracles: Meta-learning Vulnerability Detection Strategies through Adversarial Self-Play," in *Proc. 41st IEEE/ACM Int. Conf. Automated Software Engineering (ASE 2026)*, Oct. 2026, doi: 10.1145/3832783.3837512.

[23] W. Cheng, K. Sun, X. Zhang, and W. Wang, "Security Attacks on LLM-based Code Completion Tools," *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 22, pp. 23669–23677, Apr. 2025, doi: 10.1609/aaai.v39i22.34537.

Appendix B — Previously Funded Project Summary

No previously funded project. The PI (Chae) has not received cash funding or AWS Promotional Credits, directly or indirectly, from Amazon Research Awards or any other Amazon program within the past 3 years, and has no previously funded Amazon Research Award project to report. Confirmed by the PI, 2026-10-29.